TÉCNICAS ÁGILES

DE DESARROLLO DE SOFTWARE

AGILE TECHNIQUES

FOR SOFTWARE DEVELOPMENT

PRÁCTICA / *EXERCISE* #1

Introduction to Spring Boot

In this exercise we will have a first contact with **Spring Boot**, **Git** and **Docker**. The main objectives are:

- Get to know Spring Boot by running a simple *hello world application* in Spring Boot.
- Start getting to know the Thymeleaf template framework, making small modifications to the app that use a form.
- Work on a regular basis, making small commits that must be uploaded to the subject's personal repository on GitHub.
- Create a deployable app using Docker and publish the container to DockerHub

1. Software installation

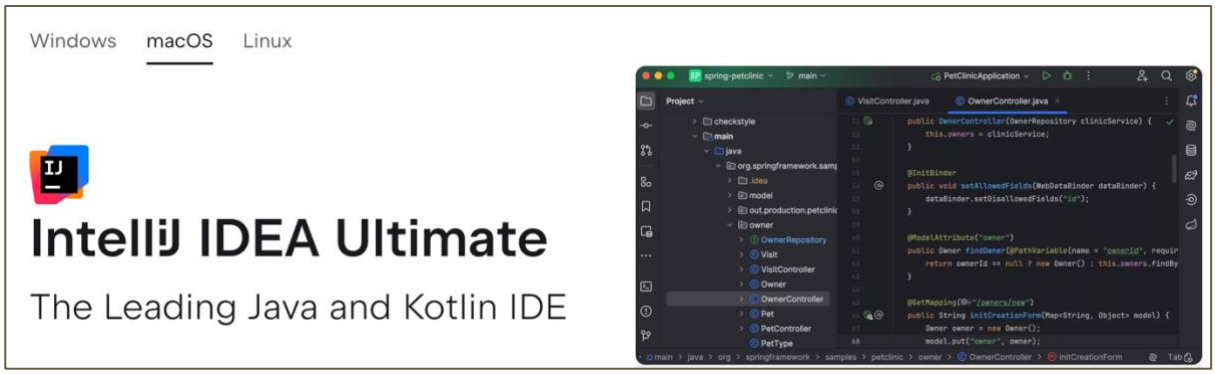
We are going to work with the terminal sometimes. On Linux or macOS we can use the terminal that comes with the system. However, on Windows you can use the **Git Bash terminal** that is installed in the **Git for Windows installation**.

It is possible to develop the exercise on any operating system but, we must install the following software:

- Git
- Java **JDK** 8 or later
- IntelliJ Ultimate

IntelliJ Ultimate

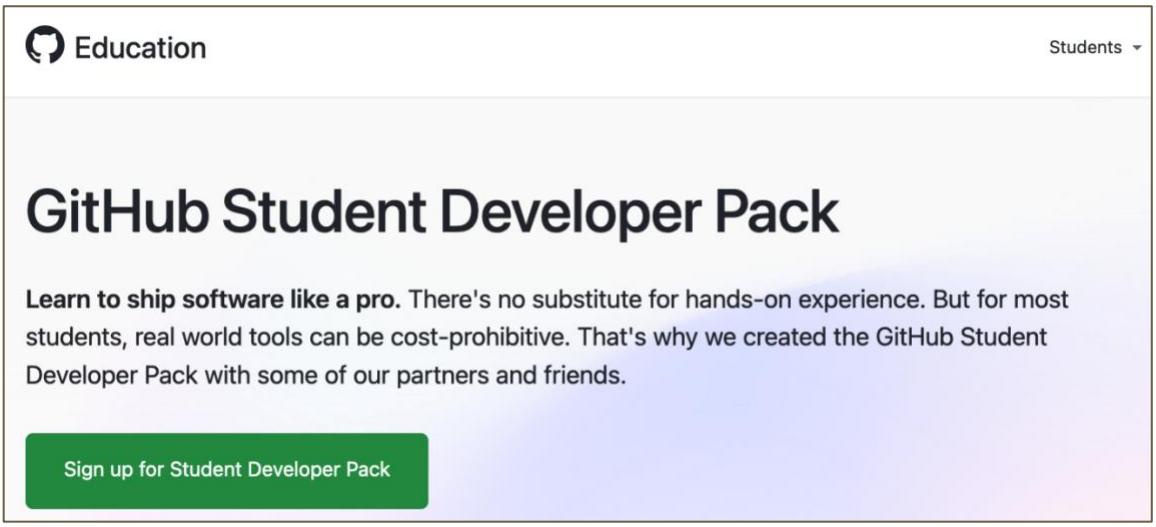
We recommend doing the exercise using the **IntelliJ Ultimate IDE**. Although it is paid, it is possible to obtain a student license using the UMH email address.



Fill out this form to get an IntelliJ Ultimate student license.

Sign up on Github

Sign up for GitHub with the UMH email and **request registration as a student** to obtain benefits such as the use of **GitHub Pro** (which includes Copilot) and to be able to request the **GitHub Student Developer Pack** with offers such as $200 on Digital Ocean servers. You will need a time-based one-time password (TOTP) application to configure **two-factor authentication** (2FA).



Basic installation on Linux

Install Git and Java:

```
$ sudo apt install git
$ sudo apt install default-jdk
```

Install **IntelliJ Ultimate**.

Basic installation on macOS

Git is already installed. Install **Java JDK *Long-Term Support (LTS)*** release.

Install **IntelliJ Ultimate.**

Basic installation on Windows

It is recommended to install **Git for Windows**. In addition, **Java** and **IntelliJ Ultimate** must be installed.

Check the installation!

It's easy to test that the installed software works. Just run from the terminal:

```
$ git --version
$ java -version (prints Java version)
```

Setting the prompt for the Git branch to appear

Bash

It is also quite useful to configure the prompt to display the branch of the Git repository in which we are. To do this, you must add $HOME/.bashrc (Linux and Git Bash Windows) or $HOME/.bash_profile (macOS with bash shell) file:

```
parse_git_branch() {
    git branch 2> /dev/null | sed -e '/^[^*]/d' -e 's/* \(.*\)/ (\1)/'
}
export PS1="\[\e[37m\]\A\[\e[m\]\[\033[32m\]\W\[\033[33m\]\$(parse_git_branch)\[\033[00m\] $ "
```

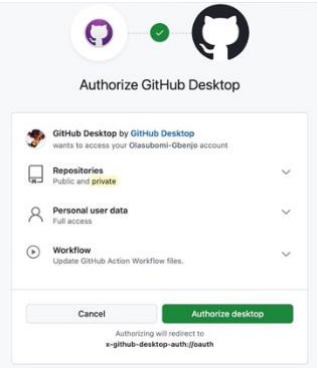
Zsh

If you work with the zsh shell that comes by default on MacOS, you must add the following to the .zshrc file:

```
parse_git_branch() {
    git branch 2> /dev/null | sed -n -e 's/^\* \(.*\)/ [\1]/p'
}
setopt PROMPT_SUBST
export PROMPT='%1~%F{green}$(parse_git_branch)%f %% '
```

Installing GitHub Desktop

If working with the terminal command line is hard for you, you can get **Github Desktop for Windows or macOS**.



After installation, don't forget to **authorize** Github Desktop to connect to your GitHub account.

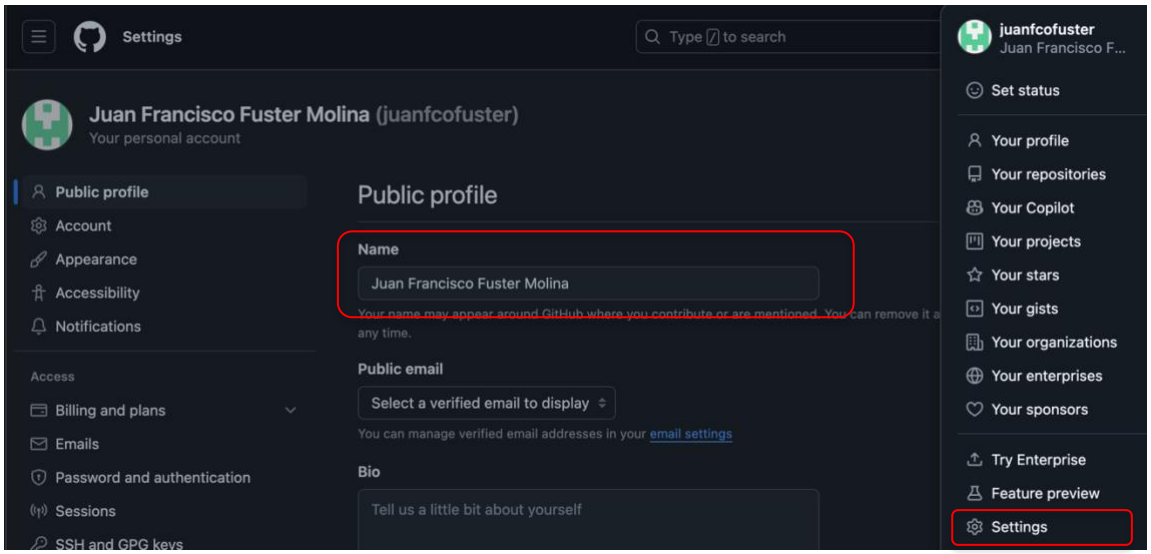
2. Creation of the GitHub repository for this exercise

To initialize the GitHub repository, you're going to work on in this exercise, you need to follow these steps:

1. Initialize your username and email on Git. The username will be the name that will appear on the commits. Put your name and surname.

```
$ git config --global user.name "Linus Torvalds"
$ git config --global user.email linus.torvalds@example.com
```

2. Create a GitHub account. You can use any username you want (or use the username you already have) but **type your first and last name correctly** into the profile using the *Settings* > *Profile* option and updating the **Name** field.



3. Create a new GitHub repository:
    a. Use GitHub Desktop to create a new GitHub repository on your local drive.
    b. Download the files you need for this exercise from the subject's online campus.
    c. Copy the files to your local repository folder.

Watch this Youtube video about how Git works:

GitHub Basics Made Easy: A Fast Beginner's Tutorial!

4. Commit all your local files to the **main brach** of your GitHub repository with the comment *Original files from the professor.*

5. It's a good idea to **create a personal access token for GitHub** if you are planning to use the command line instead of the GitHub Desktop app.

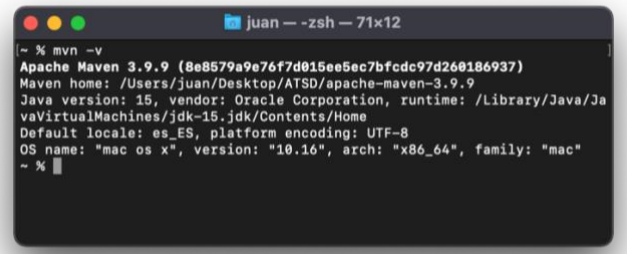
3. Demo Spring Boot app

We will do a simple exercise in which we will first launch and publish an initial app in Spring Boot and then we will add some new functionality.

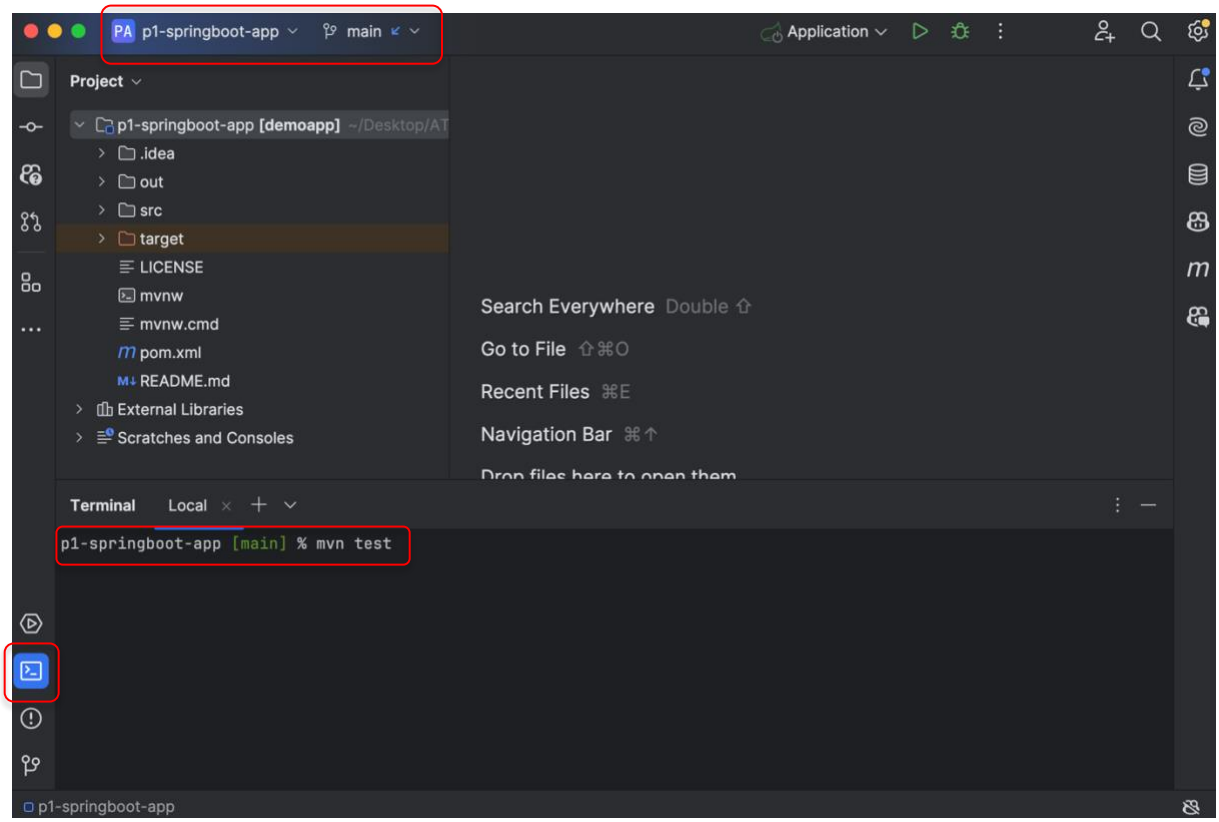
The first thing you need to do is to **install the Maven framework on your computer**. **Apache Maven** is a software project management and comprehension tool. Based on the concept of a project object model (POM), Maven can manage a project's build, reporting and documentation from a central piece of information.

Maven™

1. **Download Maven** and unzip the framework.
2. Follow the **installation instructions**.
3. Check the installation with `mvn -v` in a new command line window.



4. Import the project on IntelliJ IDE.
5. Open the IntelliJ terminal and run the test: `mvn test`



Test result should be something like this:

```
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0
```

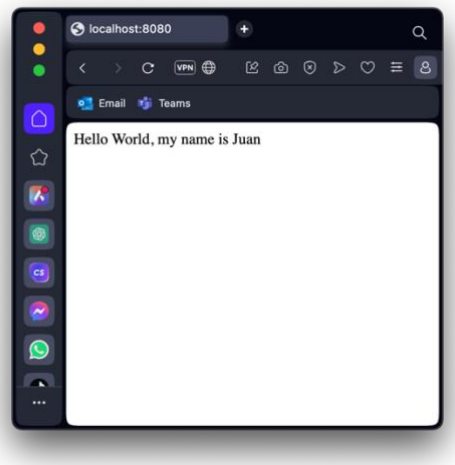
```
[INFO]
[INFO] ------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------
[INFO] Total time:  7.411 s
[INFO] Finished at: 2024-09-23T19:43:31+02:00
[INFO] ------------------------------------------------------------------
```

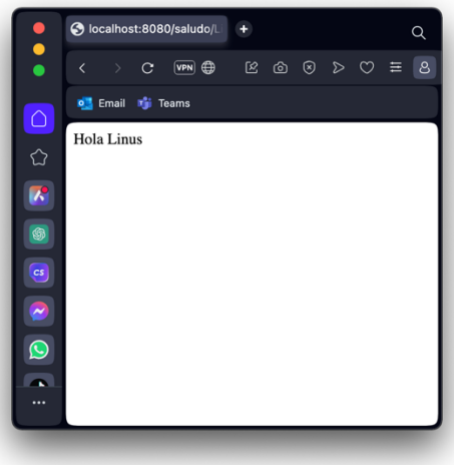
6. Run the app with the command: `mvn spring-boot:run`

```
INFO 5157 --- [           main] o.s.b.w.embedded.tomcat.TomcatWebServer  : Tomcat
initialized with port(s): 8080 (http)
INFO 5157 --- [           main] o.apache.catalina.core.StandardService   : Starting
service [Tomcat]
INFO 5157 --- [           main] org.apache.catalina.core.StandardEngine  : Starting
Servlet engine: [Apache Tomcat/9.0.78]
INFO 5157 --- [           main] o.a.c.c.C.[Tomcat].[localhost].[/]       : Initializing
Spring embedded WebApplicationContext
INFO 5157 --- [           main] w.s.c.ServletWebServerApplicationContext : Root
WebApplicationContext: initialization completed in 1045 ms
INFO 5157 --- [           main] o.s.b.w.embedded.tomcat.TomcatWebServer  : Tomcat started
on port(s): 8080 (http) with context path ''
INFO 5157 --- [           main] demoapp.Application                      : Started
Application in 2.157 seconds (JVM running for 2.506)
```

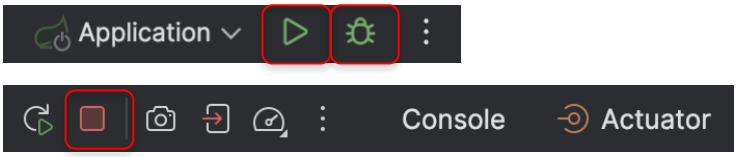
Now, the app is running. We can check the followings webpages on any web browser installed on our system:

- http://localhost:8080
- http://localhost:8080/saludo/Linus
- http://localhost:8080/saludoplantilla/Torvalds
- http://localhost:8080/saludoform

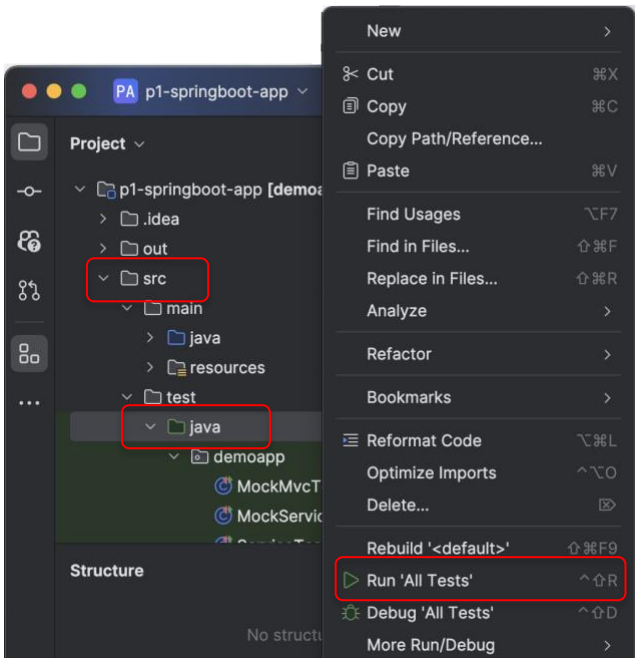




Alternatively, you can run, debug, and stop the app from the toolbar button:



Furthermore, you can run all the test from the source code:



4. Understanding the app

The `pom.xml` file declares the dependencies. Spring Boot provides ***starters*** that group together a set of common dependencies.

In the app configuration file, properties are defined that configure different aspects of the app, such as the database with which it is going to work or the port on which it must be executed. As we need to configure these properties, we will add elements to the file.

```
File: resources/application.properties
spring.application.name = demoapp
```

Controllers

The controllers define the code to be executed in response to an HTTP request. These are classes that are usually placed in the controller package and are annotated with **@Controller**.

Class SaludoController example:

```
File: src/main/java/demoapp/controller/SaludoController.java

package demoapp.controller;

import demoapp.service.SaludoService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.ResponseBody;

@Controller
public class SaludoController {
    @Autowired
    private SaludoService service;

    @RequestMapping("/saludo/{nombre}")
    public @ResponseBody String saludo(@PathVariable(value="nombre")
String nombre) {
        return service.saluda(nombre);
    }
}
```

The methods in which responses to HTTP requests are defined are annotated with annotations indicating the type of request and the URL to which it is responding.

For example, in the previous class the greeting method responds to requests addressed to the URL `/greeting/Linus`. The string `Linus` in the URL is decoded and passed in the name parameter to the method.

The method returns the HTTP response. The annotation **@ResponseBody** automatically constructs this response, adding the string returned by the service as its content.

Service classes

It is recommended that you define service classes in which the business logic of the app is implemented. The controller classes call the service classes, which are the ones that actually do all the processing.

In this way, responsibilities are separated. The controller classes are responsible for processing HTTP requests and responses, and the service classes are the ones that actually perform the business logic and return the content of the responses. If at some point you have to add a new presentation layer in which, for example, you work with JSON objects, you don't need to change the service layer, just add new controller classes.

The separation of business logic into service classes also allows tests to be carried out on Java objects, independent of the input/output formats handled by the controllers.

File: src/main/java/demoapp/service/SaludoService.java

```
package demoapp.service;
import org.springframework.stereotype.Service;
@Service
public class SaludoService {
    public String saluda(String nombre) {
        return "Hola " + nombre;
    }
}
```

Injection of dependencies in Spring

Spring Boot uses **@Autowired** annotation to inject a new object of the indicated type into the annotated variable. You can define the annotation in the variable or in the class constructor. We have used annotation in instance variables.

In the above examples we can check these notes. The sample app defines a handler and a service that returns a greeting. The service is scored with the **@Service** notation and this annotation tells Spring Boot that the class can be injected.

The controller needs to instantiate an object from the `SaludoService` class and this is done using dependency injection. Spring Boot is responsible for getting an instance and injecting it into the variable when the controller is initialized. We see it in the following code:

```
@Controller
public class SaludoController {
    @Autowired
    private SaludoService service;

    @RequestMapping("/saludo/{nombre}")
    public @ResponseBody String saludo(@PathVariable(value="nombre")
String nombre) {
        return service.saluda(nombre);
    }
}
```

By injecting dependencies, the code is more flexible and easier to test, and mocks can be defined that replace the injected objects.

Scope of injected objects

By default, the scope of all Spring annotations (@service, @controller, @component, etc.) is a **Singleton**. There is a single instance of that object that is injected into the variables.

When running a web application, the singleton that acts as the controller will receive multiple requests concurrently. Each request will go in its own Java thread, so multiple threads may be running the same controller code.

Therefore, care must be taken not to define mutable (stateful) instance variables within the controller (with the exception of objects injected with the **@Autowired** annotation), because errors could occur due to race conditions (one thread modifies the same variable that another is reading). It is convenient that all beans (controllers, services, etc.) are stateless objects.

Thymeleaf templates

The returned HTML pages can be constructed using a templating language with the **Thymeleaf framework**. Thymeleaf templates are HTML pages in which Java objects passed by controllers are entered. Templates are included in the `resources/templates` directory.

For example, the following file is a simple template that displays a greeting. The message object is a string that is passed from the controller.

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
    <h1 th:text="'Saludo: ' + ${mensaje}"> </h1>
</html>
```

The controller builds the view by returning the name of the template. And it passes to the template the objects to be displayed using a dictionary (the `model` variable that it receives as a parameter).

```
File: src/main/java/demoapp/controller/SaludoControllerPlantilla.java

package demoapp.controller;

import demoapp.service.SaludoService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RequestMapping;

@Controller
public class SaludoControllerPlantilla {
    @Autowired
    private SaludoService service;

    @RequestMapping("/saludoplantilla/{nombre}")
    public String saludo(@PathVariable(value="nombre") String nombre,
                                                        Model model) {
        // Se añade la respuesta a la clave 'mensaje' que se usa
        // en la plantilla
        model.addAttribute("mensaje", service.saluda(nombre));
        // Se llama a la plantilla 'saludo.html'
        return "saludo";
    }
}
```

Forms and validation

Spring Boot simplifies form declaration and validation by using Java classes that work as form models. For example, the app defines the **UserData** class:

```
File: src/main/java/demoapp/controller/UserData.java

package demoapp.controller;

import javax.validation.constraints.Size;
public class UserData {
    @Size(min=3, max=30)
    String nombre;
```

```
    public void setNombre(String nombre) {
        this.nombre = nombre;
    }
    public String getNombre() {
        return nombre;
    }
}
```

This is an object with the String type `nombre` field to which a validation annotation has been added **@Size** with the minimum and maximum size that it must have.

You can find more information about other validation attributes **at this link**.

The HTML form itself is defined by a `Thymeleaf` template in which an object is declared that has the same name as the name of the model class, **but with the first letter lowercase**. In our case the model class is `UserData`, so the object of the form must be called `userData`. In the form we can use any field defined in the model object. In our case we use the only field of type `String`, `name`.

File: src/main/resources/templates/formRegistro.html

```
<!DOCTYPE HTML>
<html xmlns:th="http://www.thymeleaf.org">
<body>
<form th:action="@{/saludoform}" th:object="${userData}" method="post">
    <table>
        <tr>
            <td>Nombre:</td>
            <td><input type="text" th:field="*{nombre}" /></td>
            <td th:if="${#fields.hasErrors('nombre')}" th:errors="*{nombre}"></td>
        </tr>
        <tr>
            <td><button type="submit">Enviar</button></td>
        </tr>
    </table>
</form>
</body>
</html>
```

An action is defined that will send a POST request to the `/saludoform` URL with the form data. In this case the `nombre`.

Finally, the controller that launches the form and the one that receives the POST request with the data entered are defined in the following file.

```
File: src/main/java/demoapp/controller/SaludoControllerForm.java

package demoapp.controller;

import demoapp.service.SaludoService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.validation.BindingResult;
import org.springframework.web.bind.annotation.*;

import javax.validation.Valid;

@Controller
public class SaludoControllerForm {

    @Autowired
    private SaludoService service;

    @GetMapping("/saludoform")
    // Hay que declarar un parámetro con el
    // tipo usado en el modelo del formulario (UserData)
    public String saludoForm(UserData userData) {
        return "formRegistro";
    }

    @PostMapping("/saludoform")
    public String checkPersonInfo(@ModelAttribute @Valid UserData
userData, BindingResult bindingResult, Model model) {
        if (bindingResult.hasErrors()) {
            return "formRegistro";
        }
        model.addAttribute("mensaje",
service.saluda(userData.getNombre()));
        return "saludo";
    }
}
```

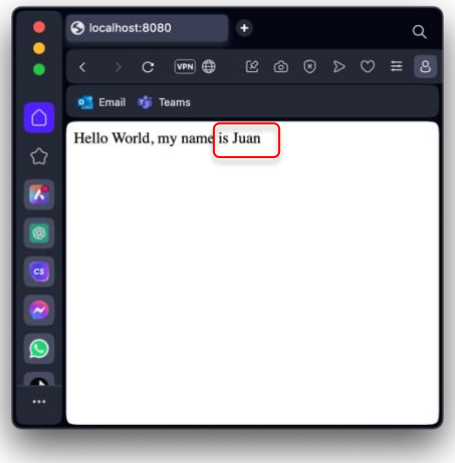
The method that receives the request in `/saludoform` simply returns the template with the form.

And the method that receives the data is the one that responds to the POST request in the URL `/saludoform` receives the form data in a Java object of the form model type (the `userData` parameter). The annotations **@ModelAttribute** and **@Valid** indicate that it is an object included in the form on which a validation has been carried out. It is detected if there has been an error in the `hasErrors()` method of the `bindingResult` that is also received as a parameter.

If there has been an error, the form is returned again to be filled out again. If there has been no error, the name entered in the form is obtained and the greeting template is passed.

5. Changing the app

To check your understanding of the app, change the greeting message given by the root controller to include **your name** instead of my name. Check that the tests pass (modify them if they don't) and that the app works correctly.



Commit these changes to your GitHub repository.

6. Dockering the app

Docker is a virtualization software that uses the compartmentalized operating system itself and allows you to manage containers (similar to virtual machines) in a much lighter and faster way than with traditional virtualization systems such as VirtualBox.

Docker machines are very efficient because they share the services of the operating system on which they run, using fewer resources than traditional virtual machines.

Docker provides a very simple system for distributing and putting software into production, since Docker machines can be distributed using repositories (such as **Docker Hub**) and run on any computer that has the Docker Engine installed.

The technology is very popular and is used in many development companies to simplify execution in multiple environments and so that containers (running Docker machines) can be configured and combined or run in clusters using tools such as **Kubernetes**.

In our case, we're going to build a Docker machine based on the demo app. We will then publish it to Docker Hub and deploy it to a host to put it into production.

1. Install **Docker Desktop**. Linux users should follow the instructions **on this page to install Docker Engine**. We'll use the command line to launch the Docker commands. The Docker Desktop app allows you to use a user interface to interact with images and containers, but it doesn't provide any functionality that isn't available on the command line.

   Once installed you can try the quick tutorial (2 minutes) from Docker Desktop to check that everything is working correctly. You can also check the installed version of Docker from the terminal:

   ```
   $ docker version
   ```

2. Create a user account in **Docker Hub**. This way you will have a repository where you can upload the images of the Docker machines you build. You will need to provide a username that you will use to post these images.

3. Create a file called `Dockerfile` (without extension) in the root directory of the Spring Boot Demoapp with the following contents:

   ```
   FROM openjdk:8-jdk-alpine
   COPY target/*.jar app.jar
   ENTRYPOINT ["java","-Djava.security.egd=file:/dev/urandom","-
   jar","/app.jar"]
   ```

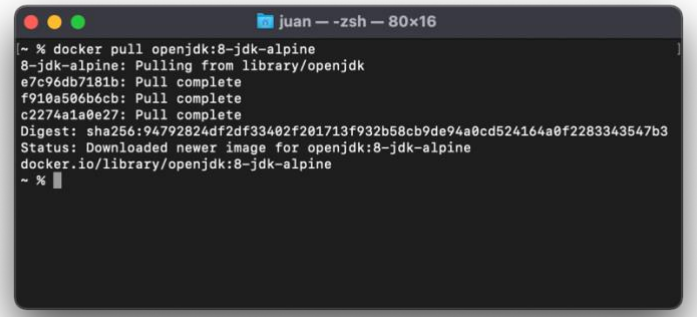
The `Dockerfile` file consists of a sequential set of instructions with which the Docker machine is built:

- `FROM`: This command indicates the base machine on which the rest of the commands are to be executed. In our case a machine of the `openjdk` organization that contains the Java Development Kit (JDK) distribution 8 and that is based on an Alpine Linux distribution. The first step in building our Docker machine is therefore to download this `openjdk:8-jdk-alpine` machine and use it as a base machine.
- `COPY`: This command indicates that a file or set of files from the host machine (the directory we are in) must be copied to the base machine. In this case, the JAR file that constitutes our app, located in the `./target` directory, is copied to the Docker machine with the name `app.jar`.
- `ENTRYPOINT`: This command indicates the command to execute when the Docker machine is started. In this case, the app (`app.jar`) is launched with the `java -jar` command.

The `-Djava.security.egd` switch causes the Java random number generator to be initialized using the `/dev/urandom` system file instead of the default `/dev/random` file. It is necessary to solve a bug that appears when running the container on servers such as those hosted by **DigitalOcean**.

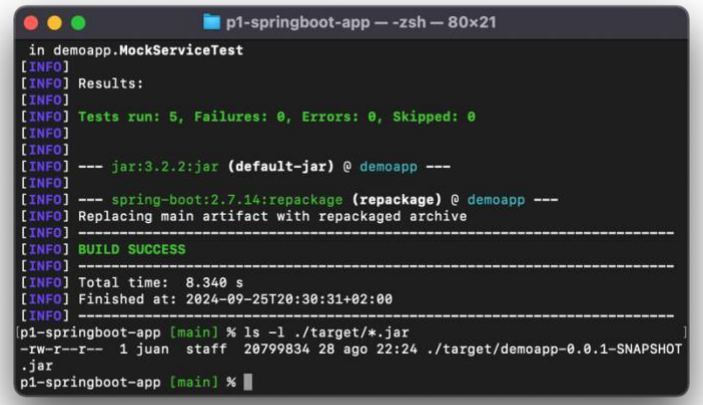
In order to get the `openjdk:8-jdk-alpine` image you can run:

```
$ docker pull openjdk:8-jdk-alpine
```



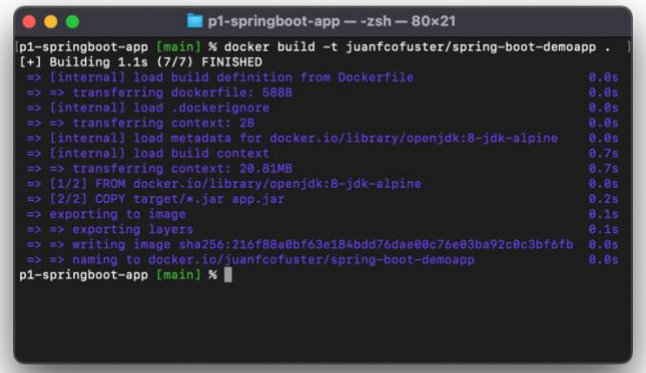
4. Recompile the app to ensure that the JAR file containing it is generated. This file is the one that we're going to dockerize.

```
$ mvn package
$ ls -l ./target/*.jar
./target/demoapp-0.0.1-SNAPSHOT.jar
```



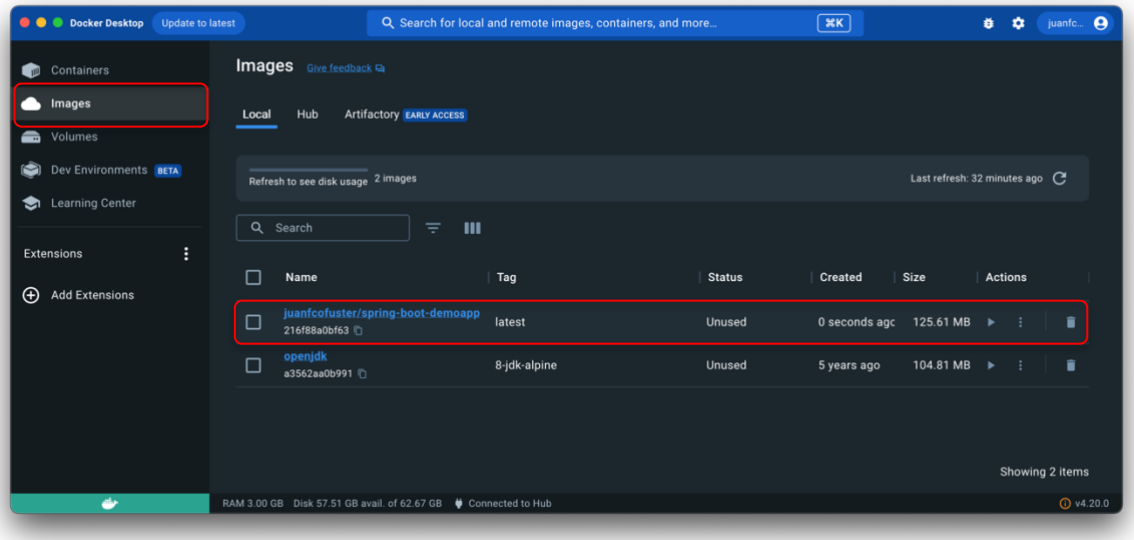
5. You can now build the Docker machine with the following command, from the root directory of the app (where the previous `Dockerfile` should be):

```
$ docker build -t <usuario-docker>/spring-boot-demoapp .
```



Check that the image was created correctly. It should appear in Docker Desktop and with the docker image `ls` command:

```
$ docker image ls
```



6. Launch an image with the app:

```
$ docker run -p 8080:8080 <usuario-docker>/spring-boot-demoapp
```

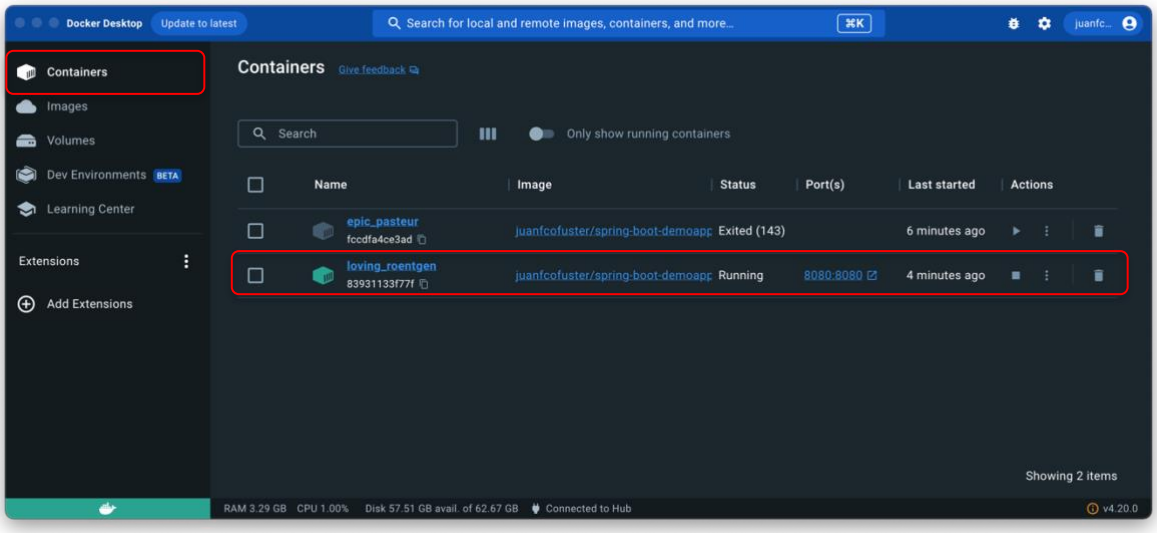
The docker run command spins up the indicated image, creating what is called a Docker container. *It is similar to run a virtual machine*. The `-p 8080:8080` parameter indicates that the container's internal port 8080 is to be mapped to the host's port 8080. In this way we will be able to connect from the host to the Spring Boot app in operation.

You'll see the console display the output messages for the Spring Boot app running in the container.

Try opening a browser and connecting to the URL localhost:8080. You should see the app's greeting message running in the container.

By pressing the shortcut Ctrl+C (Coomand+C) you can stop the container. *The effect is similar to suspending a virtual machine*. Or you can use Docker Desktop stop button.

7. Now that you've checked that the `Dockerfile` is working correctly, you need to add it to Git and upload it to the GitHub repository doing a new commit.



You can see the container ID with the command:

```
$ docker container ls -a
CONTAINER ID   IMAGE                                        COMMAND                    CREA…
83931133f77f   juanfcofuster/spring-boot-demoapp          "java -Djava.securit…"   9 minutes…
fccdfa4ce3ad   juanfcofuster/spring-boot-demoapp:latest   "java -Djava.securit…"   11 minutes…
```

You can use both the container's ID (83931133f77f) and its name to identify it.

When the container is stopped, you can restart it by:

```
$ docker container start <identificador>
```

We can also stop the container:

```
$ docker container stop <identificador>
```

And erase it definitively with:

```
$ docker container rm <identificador>
```

Other useful Docker commands are:

- `docker run -d`: Launches the container in background mode.
- `docker run --rm`: launches the container so that when it is stopped it is automatically deleted.
- `docker container logs <identificador>`: Displays the logs of the indicated container.

In the Docker Desktop app we can also perform many of these commands by interacting directly with the interface. **Try it**!

8. We're going to finish by posting the image to your Docker Hub account.
   - Go to **Docker Hub** and log in.
   - Create a **public repository** with the name `p1-spring-boot-app`. In that repository you are going to upload the image with the same name. In a Docker repository you can maintain multiple versions of the same image, using *tags*.
   - Once the repository is created, you can publish the image to it by logging in from the command line (enter your Docker Hub username and password) and using the `docker push` command:

     ```
     $ docker login
     $ docker push <usuario-docker>/spring-boot-demoapp
     ```

     You will see that the `latest` tag (default tag) is automatically assigned to the image and that it is uploaded to the repository.

     You could assign a specific tag to the image with the `docker tag` command. For example, if we wanted to fix this image with version 1.0 we could do it with the following command:

     ```
     $ docker tag <usario-docker>/spring-boot-demoapp <usuario-docker>/spring-boot-demoapp:1.0
     ```

   - Check on the repository's webpage that it has been uploaded. The repository is **public** and anyone can download the image by:

     ```
     $ docker pull <usuario-docker>/spring-boot-demoapp
     ```

   - By not indicating the label, the image tagged with latest would be downloaded. If we wanted to download a specific version, we would have to specify the tag:

     ```
     $ docker pull <usuario-docker>/spring-boot-demoapp:1.0
     ```

9. Write **a link to the public view** of the image in Docker Hub in the `README.md` file of the GitHub repository. The public hearing has the format `https://hub.docker.com/r/...../spring-boot-demoapp`.

Make sure that it is possible to access the link of your docker image **without being logged in to DockerHub**. Open a different browser where you are not logged in and check that the link is working correctly. That link is the one I'm going to use to check the correct functioning of the exercise.

7. Adding some simple functionality to the app

To show that you understand how a Spring Boot app works, you need to add some new functionality to the Hello World app. You must develop the following code:

- Palindrome: Read a word and check if it is a palindrome.
- Even number: Read a number and check if it's even.
- Square: Reads two numbers and checks if the second one is the square of the first one.
- Calculator: Reads a couple of numbers and an operation (+, -, *, /) and returns the result.
- Develop any other functionality you like (optional).

Your app must:

- Read data from a form using Thymeleaf **and perform some validation**.
- Call a **service method** that processes the read data.
- Display the result returned by the service on a Thymeleaf page.
- Include service layers and presentation layer tests (web controllers).
- Your **first and last name should appear** on the app's homepage.

We must develop the app in small commits. Each commit must compile correctly and add a **small functionality**, or it must perform a refactoring in which no new functionality is included but the code is improved.

We must upload the commits to the GitHub repository.

Small commits

For the correct performance of the exercise, you must build the app based on small commits that increase its functionality. If you have never worked in this way before, it will be somewhat complicated at first, but little by little you will get the hang of it.

The idea is that before you start writing the code you should be clear about what things you want to do.

For example, if we want to make an app that checks if a word is palindrome, we will need a function in a service that does that check. We then proceed to write code to implement it. Before recording the commit we must check that it works correctly. How do we do it? We can do this in two ways: create a simple controller that calls that function with a specific example or create a test. Once we have checked that the function is working correctly, we record the commit.

Once the service layer is finished, we must program the controller and the form to use the app. We must also do it little by little, with small commits. For example, first we can program a simple version (first commit), then we can add validations to the form (second commit) and finally we can make some improvements to the appearance of the app (third commit).

When you finish implementing all the tests (the more the better) and check that they work correctly and that the app works well locally, you must create the Docker machine with the `final` tag and upload it to your DockerHub repository.

8. Delivering the work

To deliver your work:

- Make the app in your local repository and upload the commits to GitHub as they are made, little by little.

- Update the **README.md** file with **the link to the public view** of the uploaded DockerHub image. On that page should be the image with the `final` tag. The URL format should be similar to:

  `https://hub.docker.com/r/<docker-user>/spring-boot-demoapp`.

- Document your work, adding the information to the **README.me** file, explaining briefly both **the functionality** and **the implementation** and **tests** you have added. The URL of the new functionalities (endpoints) should be included. Plus, the URL of the **repositories on GitHub and Docker Hub should also be included**. You'll need to write this documentation using **Markdown** markup language. A brief, but helpful, **Introduction to Markdown** is available on GitHub.

- Submit to this subject's online campus a ZIP file with the project directory (including the .git directory with the Git repository), after having done `./mvn clean` command to remove the compiled binaries.

The evaluation will consider:

- Continuous development (commits should be made throughout the weeks and not leave everything for the last day).

- Small commits, each commit must be functional and introduce some new element or perform some refactoring.

- Correct development of the methodology.

- Design and implementation of the code and tests of the developed features.

Penalty for performing the exercise on the last day:

The subject rewards continuous work and the completion of commits throughout the week. If all the work is done on the last day, there will be a penalty in the grade.